Script Kompresi Gambar Fish Shell

https://gemini.google.com/app/9845e569b00ca3a8

User prompt: buat script fish ini tambahkan fitur, kalau hasil kompresinya malah lebih besar dari file aslinya maka gagalkan saja. sama kalau bisa, tambahkan tag ke metadata yang menandakan bahwa file ini sudah pernah dikompresi pada tanggal sekian oleh $hostname, dan juga tag ini harus mudah dibaca oleh mesin. nah tambahkan pula supaya script ini tidak mengompres file yang dalam metadatanya sudah ada tanda pernah dikompresi #!/usr/bin/env fish # == # 1. WORKER MODE: TAHAP 1 (RAW -> JPG) # == if test "$argv[1]" = "--worker-raw" set file "$argv[2]" set dir (dirname "$file") set base (string replace -r '\.[^.]+$' '' (basename "$file")) set target_jpg "$dir/$base.jpg" # Cek apakah JPG sudah ada (untuk kasus RAW + JPG mode) if not test -f "$target_jpg"; and not test -f "$dir/$base.JPG" # Ubah RAW ke JPG kualitas tinggi (95) magick "$file" -quality 98 "$target_jpg" >/dev/null 2>&1 # Jika sukses, salin EXIF, tanggal modifikasi, lalu hapus file RAW if test -s "$target_jpg" exiftool -overwrite_original -TagsFromFile "$file" "$target_jpg" >/dev/null 2>&1 touch -r "$file" "$target_jpg" rm -f "$file" end else # Jika JPG versi asli sudah ada, langsung hapus file RAW agar hemat ruang rm -f "$file" end exit 0 end # == # 2. WORKER MODE: TAHAP 2 (JPG -> AVIF) # == if test "$argv[1]" = "--worker" set file "$argv[2]" set dir (dirname "$file") set base (string replace -r '\.[^.]+$' '' (basename "$file")) set final_avif "$dir/$base.avif" set tmp_file "$dir/$base.avif.tmp" # Konversi JPG ke AVIF (yuv 420, speed 8, q55) avifenc "$file" "$tmp_file" -q 60 --speed 8 --yuv 420 -j 1 >/dev/null 2>&1 # Jika sukses, salin EXIF & timestamp, hapus JPG asli, ganti nama .tmp ke .avif if test -s "$tmp_file" exiftool -overwrite_original -TagsFromFile "$file" "$tmp_file" >/dev/null 2>&1 touch -r "$file" "$tmp_file" rm -f "$file" mv "$tmp_file" "$final_avif" else # Jika gagal, bersihkan file .tmp jika ada rm -f "$tmp_file" end exit 0 end # == # 3. MAIN MODE: Pemindaian Folder & Pengolahan Paralel Dua Tahap # == # Cek Ketersediaan Dependensi for cmd in avifenc parallel exiftool magick if not command -q $cmd echo "Error: Perintah '$cmd' tidak ditemukan. Silakan install terlebih dahulu." return 1 end end echo "=== Memulai Pemindaian & Otomasi Konversi Gambar ke AVIF ===" echo "" # Catat Ukuran Folder Sebelum Pemrosesan set size_before_b (du -sb . | string split -f1 \t) # Dapatkan lokasi file skrip ini agar bisa dipanggil kembali oleh Parallel set self_script (status filename) # ---------- # TAHAP 1: Konversi Seluruh File RAW ke JPG & Hapus RAW Asli # ---------- echo "==> [Tahap 1/2] Memproses & Mengonversi File RAW ke JPG..." find . -type f \(-iname ".cr2" -o -iname "*.cr3" -o -iname "*.arw" -o -iname "*.srf" -o -iname "*.sr2" -o -iname "*.nef" -o -iname "*.nrw" -o -iname "*.dng" -o -iname "*.raf" -o -iname "*.rw2" -o -iname "*.orf" -o -iname "*.pef" \) | parallel --bar -j 100% fish "$self_script" --worker-raw {} # ---------- # TAHAP 2: Konversi Seluruh File JPG ke AVIF (Termasuk Output Tahap 1) # ---------- echo "" echo "==> [Tahap 2/2] Memproses & Mengonversi Seluruh JPG ke AVIF..." find . -type f \(-iname "*.jpg" -o -iname "*.jpeg" \) | parallel --bar -j 100% fish "$self_script" --worker {} # ---------- # LAPORAN RINGKASAN UKURAN FOLDER # ---------- set size_after_b (du -sb . | string split -f1 \t) set before_mb (math -s 2 "$size_before_b / 1024 / 1024") set after_mb (math -s 2 "$size_after_b / 1024 / 1024") set saved_mb (math -s 2 "$before_mb - $after_mb") if test $size_before_b -gt 0 set saved_pct (math -s 2 "($size_before_b - $size_after_b) / $size_before_b * 100") else set saved_pct 0 end echo "" echo "==" echo " HASIL RINGKASAN KONVERSI " echo "==" echo "Ukuran Folder Awal : $before_mb MB" echo "Ukuran Folder Akhir : $after_mb MB" echo "Total Hemat Ukuran : $saved_mb MB ($saved_pct%)" echo "==="*

Response: Berikut adalah pembaruan script Fish yang telah dilengkapi dengan pengecekan ukuran file, penandaan metadata berformat *machine-readable* (ISO 8601 + $hostname), serta mekanisme *skip* otomatis jika file sudah pernah melewati proses kompresi sebelumnya.

```
#!/usr/bin/env fish

# ============================================================================
# 1. WORKER MODE: TAHAP 1 (RAW → JPG)
# ============================================================================
if test "$argv[1]" = "--worker-raw"
    set file "$argv[2]"
    set dir (dirname "$file")
    set base (string replace -r '\.[^.]+$' '' (basename "$file"))
    set target_jpg "$dir/$base.jpg"

    # Cek apakah file RAW ini entah bagaimana sudah memiliki tag kompresi
    set existing_tag (exiftool -s3 -UserComment "$file" 2>/dev/null)
    if string match -q "*COMPRESSED_BY_SCRIPT*" "$existing_tag"
        exit 0
    end

    # Cek apakah JPG sudah ada (untuk kasus RAW + JPG mode)
    if not test -f "$target_jpg"; and not test -f "$dir/$base.JPG"
        # Ubah RAW ke JPG kualitas tinggi (98)
        magick "$file" -quality 98 "$target_jpg" >/dev/null 2>&1

        if test -s "$target_jpg"
            # Salin EXIF terlebih dahulu agar ukuran akhir akurat sebelum dibandingkan
            exiftool -overwrite_original -TagsFromFile "$file" "$target_jpg" >/dev/null 2>&1

            set orig_size (stat -c %s "$file")
            set new_size (stat -c %s "$target_jpg")

            # Cek apakah ukuran JPG baru lebih kecil dari file RAW asli
            if test $new_size -lt $orig_size
                touch -r "$file" "$target_jpg"
                rm -f "$file"
            else
                # Gagalkan jika hasil kompresi lebih besar atau sama
                rm -f "$target_jpg"
            end
        else
            rm -f "$target_jpg"
        end
    else
        # Jika JPG versi asli sudah ada, langsung hapus file RAW agar hemat ruang
        rm -f "$file"
    end
    exit 0
end

# ============================================================================
# 2. WORKER MODE: TAHAP 2 (JPG → AVIF)
# ============================================================================
if test "$argv[1]" = "--worker"
    set file "$argv[2]"
    set dir (dirname "$file")
    set base (string replace -r '\.[^.]+$' '' (basename "$file"))

    set final_avif "$dir/$base.avif"
    set tmp_file "$dir/$base.avif.tmp"

    # 1. Cek apakah file JPG sudah pernah ditandai oleh script sebelumnya
    set existing_tag (exiftool -s3 -UserComment "$file" 2>/dev/null)
    if string match -q "*COMPRESSED_BY_SCRIPT*" "$existing_tag"
        exit 0
    end

    # 2. Konversi JPG ke AVIF (yuv 420, speed 8, q60)
    avifenc "$file" "$tmp_file" -q 60 --speed 8 --yuv 420 -j 1 >/dev/null 2>&1

    if test -s "$tmp_file"
        # Siapkan variabel metadata yang mudah dibaca mesin (Format ISO 8601)
        set comp_date (date -Iseconds)
        set host_name (hostname)
        set meta_payload "COMPRESSED_BY_SCRIPT=true;DATE=$comp_date;HOST=$host_name"

        # 3. Salin EXIF dari JPG asli sekaligus suntikkan tag metadata ke file .tmp
        # Penambahan dilakukan SEBELUM cek ukuran agar penambahan byte EXIF ikut terhitung
        exiftool -overwrite_original -TagsFromFile "$file" \
            -UserComment="$meta_payload" \
```

```
            -XMP:CreatorTool="fish-avif-compressor ($host_name)" \
            "$tmp_file" >/dev/null 2>&1

        set orig_size (stat -c %s "$file")
        set new_size (stat -c %s "$tmp_file")

        # 4. Bandingkan ukuran: Hanya lanjutkan jika file baru benar-benar lebih kecil
        if test $new_size -lt $orig_size
            touch -r "$file" "$tmp_file"
            rm -f "$file"
            mv "$tmp_file" "$final_avif"
        else
            # Gagalkan konversi jika ukuran AVIF ≥ JPG asli
            rm -f "$tmp_file"

            # Opsional tapi penting: Tandai JPG asli bahwa ia sudah pernah dicoba
            # dikompresi agar pemindaian berikutnya tidak membuang CPU untuk file ini lagi
            exiftool -overwrite_original -UserComment="$meta_payload;STATUS=SKIPPED_LARGER" "$file" >/dev/null 2>&1
            # Kembalikan timestamp modifikasi JPG asli
            # (Namun karena kita baru saja mengubah exif, kita simpan timestamp sebelumnya jika perlu)
        end
    else
        # Jika gagal encode, bersihkan file .tmp
        rm -f "$tmp_file"
    end
    exit 0
end

# ==============================================================================
# 3. MAIN MODE: Pemindaian Folder & Pengolahan Paralel Dua Tahap
# ==============================================================================

# Cek Ketersediaan Dependensi
for cmd in avifenc parallel exiftool magick stat date hostname
    if not command -q $cmd
        echo "Error: Perintah '$cmd' tidak ditemukan. Silakan install terlebih dahulu."
        exit 1
    end
end

echo "≡≡≡ Memulai Pemindaian & Otomasi Konversi Gambar ke AVIF ≡≡≡"
echo ""

# Catat Ukuran Folder Sebelum Pemrosesan
set size_before_b (du -sb . | string split -f1 \t)

# Dapatkan lokasi file skrip ini agar bisa dipanggil kembali oleh Parallel
set self_script (status filename)

# ------------------------------------------------------------------------------
# TAHAP 1: Konversi Seluruh File RAW ke JPG & Hapus RAW Asli
# ------------------------------------------------------------------------------
echo "⟹ [Tahap 1/2] Memproses & Mengonversi File RAW ke JPG..."
find . -type f \( \
    -iname "*.cr2" -o -iname "*.cr3" -o \
    -iname "*.arw" -o -iname "*.srf" -o -iname "*.sr2" -o \
    -iname "*.nef" -o -iname "*.nrw" -o \
    -iname "*.dng" -o -iname "*.raf" -o \
    -iname "*.rw2" -o -iname "*.orf" -o -iname "*.pef" \
\) | parallel --bar -j 100% fish "$self_script" --worker-raw {}

# ------------------------------------------------------------------------------
# TAHAP 2: Konversi Seluruh File JPG ke AVIF (Termasuk Output Tahap 1)
# ------------------------------------------------------------------------------
echo ""
echo "⟹ [Tahap 2/2] Memproses & Mengonversi Seluruh JPG ke AVIF..."
find . -type f \( -iname "*.jpg" -o -iname "*.jpeg" \) | parallel --bar -j 100% fish "$self_script" --worker {}

# ------------------------------------------------------------------------------
# LAPORAN RINGKASAN UKURAN FOLDER
# ------------------------------------------------------------------------------
set size_after_b (du -sb . | string split -f1 \t)

set before_mb (math -s 2 "$size_before_b / 1024 / 1024")
set after_mb (math -s 2 "$size_after_b / 1024 / 1024")
set saved_mb (math -s 2 "$before_mb - $after_mb")

if test $size_before_b -gt 0
    set saved_pct (math -s 2 "($size_before_b - $size_after_b) / $size_before_b * 100")
else
    set saved_pct 0
end

echo ""
echo "=========================================="
echo "        HASIL RINGKASAN KONVERSI          "
echo "=========================================="
echo "Ukuran Folder Awal  : $before_mb MB"
echo "Ukuran Folder Akhir : $after_mb MB"
echo "Total Hemat Ukuran  : $saved_mb MB ($saved_pct%)"
echo "=========================================="
```

Detail Perubahan & Penjelasan Teknis

1. **Format Metadata *Machine-Readable* (`EXIF:UserComment`)**
   - Tag disimpan ke dalam field standar `UserComment` (yang didukung penuh baik oleh container JPG maupun AVIF) menggunakan struktur *key-value* dengan delimiter titik koma: `COMPRESSED_BY_SCRIPT=true;DATE=2026-11-09T08:38:00+07:00;HOST=nama-host`
   - Untuk mengecek atau mem-parsing tag ini dari script lain (misal dengan `grep`, `awk`, atau `string split` di Fish), gunakan perintah:

     ```
     exiftool -s3 -UserComment nama_file.avif
     ```

     Opsi `-s3` pada `exiftool` langsung mencetak isi *value*-nya saja tanpa nama label di depannya, sehingga sangat mudah di-pipe ke program lain.
2. **Pengecekan Ukuran Setelah Injeksi EXIF (`stat -c %s`)**
   - Penyalinan metadata (`exiftool -TagsFromFile`) dilakukan **sebelum** pengecekan ukuran file (`if test $new_size -lt $orig_size`).
   - Alasannya: data EXIF/XMP (terutama jika ada *thumbnail preview* atau profil warna ICC dari kamera) bisa berukuran puluhan hingga ratusan KB. Jika dibandingkan sebelum EXIF disalin, ada kemungkinan file `.tmp` terlihat lebih kecil, tapi setelah disalin EXIF ukurannya justru membengkak melebihi file asli.
3. **Penanganan File JPG yang Gagal Kompresi (Anti-Loop)**

- Jika sebuah file JPG sudah sangat terkompresi dan hasil konversi `.avif`-nya ternyata lebih besar, script akan menghapus `.avif.tmp` dan membiarkan `.jpg` asli tetap utuh.
- Agar pada eksekusi script di masa mendatang file `.jpg` tersebut tidak dikompresi ulang secara sia-sia, script menyuntikkan tag `COMPRESSED_BY_SCRIPT=true;...;STATUS=SKIPPED_LARGER` ke dalam JPG asli tersebut. Jika Anda ingin mempertahankan *timestamp* asli JPG saat kondisi batal ini terjadi, tambahkan opsi `-P` (*preserve date/time*) pada perintah `exiftool` di blok `else` tersebut:

  ```
  exiftool -P -overwrite_original -UserComment="$meta_payload;STATUS=SKIPPED_LARGER" "$file" >/dev/null 2>&1
  ```

4. **Perbaikan Kecil pada Validasi Dependensi**
   - Mengubah `return 1` menjadi `exit 1` pada pengecekan dependensi di bagian Main Mode, karena di Fish `return` hanya valid di dalam blok `function` atau file yang di-*source*, bukan saat dieksekusi langsung sebagai script mandiri.

User prompt: tambahkan fitur supaya misal aku ctrl+c ditengah jalan, dia masih nyimpen progres gitu, jadi misal mau dilanjut itu ga usah ngescan dari awal dan bisa pick up where we left

Response: Untuk menangani interupsi (`Ctrl+C`) tanpa perlu memindai ulang folder atau memanggil `exiftool` pada file yang sudah diproses, kita bisa memanfaatkan fitur bawaan **GNU Parallel** (`--joblog` dan `--resume`) yang dikombinasikan dengan *file antrean statis* (queue file) serta *signal trap* untuk membersihkan file `.tmp` yang setengah jadi.

Berikut adalah pembaruan script Fish dengan sistem *checkpoint & resume* otomatis:

```
#!/usr/bin/env fish

# ==============================================================================
# 1. WORKER MODE: TAHAP 1 (RAW → JPG)
# ==============================================================================
if test "$argv[1]" = "--worker-raw"
    set file "$argv[2]"

    # Jika file sudah tidak ada (misal terhapus di sesi sebelumnya), lewati
    if not test -f "$file"
        exit 0
    end

    set dir (dirname "$file")
    set base (string replace -r '\.[^.]+$' '' (basename "$file"))
    set target_jpg "$dir/$base.jpg"
    set tmp_jpg "$dir/$base.jpg.tmp"

    # Bersihkan file .tmp jika worker dihentikan paksa (Ctrl+C / SIGINT / SIGTERM)
    function cleanup_raw --on-signal INT --on-signal TERM --inherit-variable tmp_jpg
        rm -f "$tmp_jpg"
        exit 130
    end

    # Cek apakah file RAW sudah memiliki tag kompresi
    set existing_tag (exiftool -s3 -UserComment "$file" 2>/dev/null)
    if string match -q "*COMPRESSED_BY_SCRIPT*" "$existing_tag"
        exit 0
    end

    # Cek apakah JPG sudah ada (untuk kasus RAW + JPG mode)
    if not test -f "$target_jpg"; and not test -f "$dir/$base.JPG"
        # Render ke file .tmp terlebih dahulu agar aman jika di-Ctrl+C
        magick "$file" -quality 98 "$tmp_jpg" >/dev/null 2>&1

        if test -s "$tmp_jpg"
            exiftool -overwrite_original -TagsFromFile "$file" "$tmp_jpg" >/dev/null 2>&1

            set orig_size (stat -c %s "$file")
            set new_size (stat -c %s "$tmp_jpg")

            if test $new_size -lt $orig_size
                touch -r "$file" "$tmp_jpg"
                mv "$tmp_jpg" "$target_jpg"
                rm -f "$file"
            else
                rm -f "$tmp_jpg"
                set comp_date (date -Iseconds)
                set host_name (hostname)
                exiftool -P -overwrite_original -UserComment="COMPRESSED_BY_SCRIPT=true;DATE=$comp_date;HOST=$host_name;STATUS=SKIPPED_LARGER" "$file" >/dev/null 2>&1
            end
        else
            rm -f "$tmp_jpg"
        end
    else
        # Jika JPG versi asli sudah ada, langsung hapus file RAW
        rm -f "$file"
    end
    exit 0
end

# ==============================================================================
# 2. WORKER MODE: TAHAP 2 (JPG → AVIF)
# ==============================================================================
if test "$argv[1]" = "--worker"
    set file "$argv[2]"

    if not test -f "$file"
        exit 0
    end

    set dir (dirname "$file")
    set base (string replace -r '\.[^.]+$' '' (basename "$file"))

    set final_avif "$dir/$base.avif"
    set tmp_file "$dir/$base.avif.tmp"

    # Bersihkan file .tmp jika worker terkena Ctrl+C di tengah proses encoding
    function cleanup_avif --on-signal INT --on-signal TERM --inherit-variable tmp_file
        rm -f "$tmp_file"
        exit 130
    end

    # 1. Cek apakah file JPG sudah pernah ditandai oleh script sebelumnya
    set existing_tag (exiftool -s3 -UserComment "$file" 2>/dev/null)
    if string match -q "*COMPRESSED_BY_SCRIPT*" "$existing_tag"
        exit 0
    end

    # 2. Konversi JPG ke AVIF (yuv 420, speed 8, q60)
    avifenc "$file" "$tmp_file" -q 60 --speed 8 --yuv 420 -j 1 >/dev/null 2>&1

    if test -s "$tmp_file"
```

```
        set comp_date (date -Iseconds)
        set host_name (hostname)
        set meta_payload "COMPRESSED_BY_SCRIPT=true;DATE=$comp_date;HOST=$host_name"

        # 3. Salin EXIF & suntikkan tag metadata ke file .tmp
        exiftool -overwrite_original -TagsFromFile "$file" \
            -UserComment="$meta_payload" \
            -XMP:CreatorTool="fish-avif-compressor ($host_name)" \
            "$tmp_file" >/dev/null 2>&1

        set orig_size (stat -c %s "$file")
        set new_size (stat -c %s "$tmp_file")

        # 4. Bandingkan ukuran
        if test $new_size -lt $orig_size
            touch -r "$file" "$tmp_file"
            mv "$tmp_file" "$final_avif"
            rm -f "$file"
        else
            rm -f "$tmp_file"
            exiftool -P -overwrite_original -UserComment="$meta_payload;STATUS=SKIPPED_LARGER" "$file" >/dev/null 2>&1
        end
    else
        rm -f "$tmp_file"
    end
    exit 0
end

# ==============================================================================
# 3. MAIN MODE: Pemindaian, State Management (Resume), & Pengolahan Paralel
# ==============================================================================

# Cek Ketersediaan Dependensi
for cmd in avifenc parallel exiftool magick stat date hostname
    if not command -q $cmd
        echo "Error: Perintah '$cmd' tidak ditemukan. Silakan install terlebih dahulu."
        exit 1
    end
end

set self_script (status filename)

# Direktori tersembunyi untuk menyimpan state progres
set state_dir ".avif_compress_state"
set raw_list "$state_dir/raw_queue.txt"
set raw_log "$state_dir/raw_joblog.txt"
set jpg_list "$state_dir/jpg_queue.txt"
set jpg_log "$state_dir/jpg_joblog.txt"
set init_size_file "$state_dir/size_before.txt"

# Support flag --reset jika ingin paksa scan ulang dari nol
if test "$argv[1]" = "--reset"
    echo "Meriset state progres sebelumnya..."
    rm -rf "$state_dir"
end

# Bersihkan sisa file .tmp yang mungkin tertinggal dari sesi crash sebelumnya
find . -type f \( -name "*.avif.tmp" -o -name "*.jpg.tmp" \) -delete 2>/dev/null

echo "≡≡≡ Memulai Otomasi Konversi Gambar ke AVIF ≡≡≡"

if test -d "$state_dir"
    echo "[RESUME MODE] Progres sesi sebelumnya ditemukan! Melanjutkan dari titik terakhir..."
    echo "(Gunakan argumen '--reset' jika ingin memindai ulang dari awal)"
else
    mkdir -p "$state_dir"
end
echo ""

# Catat Ukuran Folder Awal (Hanya dicatat saat pertama kali jalan agar kalkulasi akurat saat resume)
if not test -f "$init_size_file"
    du -sb --exclude="$state_dir" . | string split -f1 \t > "$init_size_file"
end
set size_before_b (cat "$init_size_file")

# Tangkap sinyal Ctrl+C di Main Process agar menampilkan pesan panduan resume
function on_main_interrupt --on-signal INT --on-signal TERM
    echo ""
    echo "================================================================="
    echo "[DIBERHENTIKAN] Proses dihentikan oleh pengguna (Ctrl+C)."
    echo "Progres telah disimpan di '$state_dir'."
    echo "Jalankan ulang script ini kapan saja untuk melanjutkan progres!"
    echo "================================================================="
    exit 130
end

# ------------------------------------------------------------------------------
# TAHAP 1: Konversi Seluruh File RAW ke JPG
# ------------------------------------------------------------------------------
if not test -f "$state_dir/stage1.done"
    if not test -f "$raw_list"
        echo "⟹ [Tahap 1/2] Memindai daftar file RAW..."
        find . -type f \( \
            -iname "*.cr2" -o -iname "*.cr3" -o \
            -iname "*.arw" -o -iname "*.srf" -o -iname "*.sr2" -o \
            -iname "*.nef" -o -iname "*.nrw" -o \
            -iname "*.dng" -o -iname "*.raf" -o \
            -iname "*.rw2" -o -iname "*.orf" -o -iname "*.pef" \
        \) > "$raw_list"
    else
        echo "⟹ [Tahap 1/2] Memuat daftar file RAW dari sesi sebelumnya..."
    end

    if test -s "$raw_list"
        echo "⟹ [Tahap 1/2] Memproses & Mengonversi File RAW ke JPG..."
        # --joblog dan --resume membuat parallel melewati baris yang sudah selesai
        parallel --bar -j 100% --joblog "$raw_log" --resume \
            fish "$self_script" --worker-raw {} :::: "$raw_list"

        if test $status -ne 0
            on_main_interrupt
```

```
        end
    else
        echo "==> [Tahap 1/2] Tidak ada file RAW yang ditemukan."
    end

    # Tandai Tahap 1 sudah 100% selesai
    touch "$state_dir/stage1.done"
else
    echo "==> [Tahap 1/2] Sudah selesai pada sesi sebelumnya. Melewati..."
end

# ------------------------------------------------------------------------------
# TAHAP 2: Konversi Seluruh File JPG ke AVIF (Termasuk Output Tahap 1)
# ------------------------------------------------------------------------------
echo ""
if not test -f "$state_dir/stage2.done"
    if not test -f "$jpg_list"
        echo "==> [Tahap 2/2] Memindai daftar file JPG..."
        find . -type f \( -iname "*.jpg" -o -iname "*.jpeg" \) > "$jpg_list"
    else
        echo "==> [Tahap 2/2] Memuat daftar file JPG dari sesi sebelumnya..."
    end

    if test -s "$jpg_list"
        echo "==> [Tahap 2/2] Memproses & Mengonversi Seluruh JPG ke AVIF..."
        parallel --bar -j 100% --joblog "$jpg_log" --resume \
            fish "$self_script" --worker {} :::: "$jpg_list"

        if test $status -ne 0
            on_main_interrupt
        end
    else
        echo "==> [Tahap 2/2] Tidak ada file JPG yang ditemukan."
    end

    touch "$state_dir/stage2.done"
end

# ------------------------------------------------------------------------------
# LAPORAN RINGKASAN UKURAN FOLDER & BERSIHKAN STATE
# ------------------------------------------------------------------------------
set size_after_b (du -sb --exclude="$state_dir" . | string split -f1 \t)

# Hapus folder state karena seluruh pekerjaan sudah tuntas 100%
rm -rf "$state_dir"

set before_mb (math -s 2 "$size_before_b / 1024 / 1024")
set after_mb (math -s 2 "$size_after_b / 1024 / 1024")
set saved_mb (math -s 2 "$before_mb - $after_mb")

if test $size_before_b -gt 0
    set saved_pct (math -s 2 "($size_before_b - $size_after_b) / $size_before_b * 100")
else
    set saved_pct 0
end

echo ""
echo "========================================="
echo "        HASIL RINGKASAN KONVERSI         "
echo "========================================="
echo "Ukuran Folder Awal  : $before_mb MB"
echo "Ukuran Folder Akhir : $after_mb MB"
echo "Total Hemat Ukuran  : $saved_mb MB ($saved_pct%)"
echo "========================================="
```

Cara Kerja Fitur *Resume* Ini:

1. **Penyimpanan Antrean Statis (`.avif_compress_state/`)**
   - Alih-alih langsung mem-pipe `find` ke `parallel`, hasil `find` disimpan dulu ke dalam file teks (`raw_queue.txt` dan `jpg_queue.txt`).
   - Saat kamu menekan `Ctrl+C` lalu menjalankan ulang script, script melihat bahwa file antrean tersebut sudah ada, sehingga **tidak perlu menjalankan `find` ulang** dari awal.
2. **Tracking Eksekusi dengan `parallel --joblog` & `--resume`**
   - GNU Parallel mencatat setiap nomor urut tugas (beserta *exit code*-nya) yang berhasil diselesaikan ke dalam file `raw_joblog.txt` dan `jpg_joblog.txt`.
   - Ketika dilanjutkan, flag `--resume` membuat GNU Parallel membaca *joblog* tersebut dan langsung melompati ratusan/ribuan file yang sudah diproses sebelumnya dalam hitungan milidetik—tanpa perlu memanggil worker atau `exiftool` sama sekali untuk file-file tersebut.
   - Pekerjaan yang terpotong tepat saat `Ctrl+C` ditekan tidak akan tercatat selesai di *joblog* (atau memiliki exit code non-nol), sehingga otomatis diulang dengan bersih saat dilanjutkan.
3. **Pembersihan File Setengah Jadi (`--on-signal INT`)**
   - Baik Tahap 1 maupun Tahap 2 kini menulis ke file `.tmp` terlebih dahulu (`*.jpg.tmp` dan `*.avif.tmp`).
   - Jika `Ctrl+C` ditekan saat `avifenc` atau `magick` sedang merender di tengah jalan, *signal handler* di dalam worker akan langsung menghapus file `.tmp` yang korup/setengah jadi tersebut sebelum keluar, dan file asli tetap utuh.
4. **Akurasi Ukuran Awal & Opsi Reset**
   - Ukuran awal folder (`size_before.txt`) disimpan pada sesi pertama, sehingga meskipun kamu *pause* dan *resume* berkali-kali, laporan penghematan MB di akhir tetap dihitung dari ukuran asli sebelum kompresi pertama dimulai.
   - Saat semua tahap tuntas 100%, folder `.avif_compress_state` otomatis dihapus bersih. Jika di tengah jalan kamu berubah pikiran dan ingin memindai ulang dari nol, cukup jalankan `./nama_script.fish --reset`.

User prompt: oke tambahkan fitur argumen script jadi misal dipanggil dengan argumen --fresh itu dia bakal start dari awal meskipun ada file log job sebelumnya, buat jaga jaga misal kita habis melakukan perubahan pada folder. tambahkan juga argumen -v untuk meningkatkan verbosity.

Response: Berikut adalah pembaruan script Fish yang menggunakan fungsi bawaan `argparse` untuk menangani argumen `--fresh`, `-v` / `--verbose`, serta `-h` / `--help` secara rapi dan fleksibel (bisa digabung seperti `./script.fish --fresh -v`).

```
#!/usr/bin/env fish

# ==============================================================================
# 1. WORKER MODE: TAHAP 1 (RAW → JPG)
# ==============================================================================
if test "$argv[1]" = "--worker-raw"
    set file "$argv[2]"
    set is_verbose "$argv[3]"

    # Jika file sudah tidak ada (misal terhapus di sesi sebelumnya), lewati
    if not test -f "$file"
```

```
        exit 0
    end

    set dir (dirname "$file")
    set base (string replace -r '\.[^.]+$' '' (basename "$file"))
    set target_jpg "$dir/$base.jpg"
    set tmp_jpg "$dir/$base.jpg.tmp"

    # Bersihkan file .tmp jika worker dihentikan paksa (Ctrl+C / SIGINT / SIGTERM)
    function cleanup_raw --on-signal INT --on-signal TERM --inherit-variable tmp_jpg
        rm -f "$tmp_jpg"
        exit 130
    end

    # Cek apakah file RAW sudah memiliki tag kompresi
    set existing_tag (exiftool -s3 -UserComment "$file" 2>/dev/null)
    if string match -q "*COMPRESSED_BY_SCRIPT*" "$existing_tag"
        if test "$is_verbose" = "1"
            echo "[SKIP-TAG] $file (Sudah pernah diproses sebelumnya)"
        end
        exit 0
    end

    # Cek apakah JPG sudah ada (untuk kasus RAW + JPG mode)
    if not test -f "$target_jpg"; and not test -f "$dir/$base.JPG"
        # Render ke file .tmp terlebih dahulu agar aman jika di-Ctrl+C
        magick "$file" -quality 98 "$tmp_jpg" >/dev/null 2>&1

        if test -s "$tmp_jpg"
            exiftool -overwrite_original -TagsFromFile "$file" "$tmp_jpg" >/dev/null 2>&1

            set orig_size (stat -c %s "$file")
            set new_size (stat -c %s "$tmp_jpg")

            if test $new_size -lt $orig_size
                touch -r "$file" "$tmp_jpg"
                mv "$tmp_jpg" "$target_jpg"
                rm -f "$file"
                if test "$is_verbose" = "1"
                    set orig_kb (math -s 1 "$orig_size / 1024")
                    set new_kb (math -s 1 "$new_size / 1024")
                    echo "[OK-RAW]   $file → $target_jpg ($orig_kb KB → $new_kb KB)"
                end
            else
                rm -f "$tmp_jpg"
                set comp_date (date -Iseconds)
                set host_name (hostname)
                exiftool -P -overwrite_original -UserComment="COMPRESSED_BY_SCRIPT=true;DATE=$comp_date;HOST=$host_name;STATUS=SKIPPED_LARGER" "$file" >/dev/null 2>&1
                if test "$is_verbose" = "1"
                    set orig_kb (math -s 1 "$orig_size / 1024")
                    set new_kb (math -s 1 "$new_size / 1024")
                    echo "[BATAL]    $file (JPG $new_kb KB ≥ RAW $orig_kb KB, dilewati)"
                end
            end
        else
            rm -f "$tmp_jpg"
            if test "$is_verbose" = "1"
                echo "[GAGAL]    $file (Gagal merender RAW ke JPG)"
            end
        end
    else
        # Jika JPG versi asli sudah ada, langsung hapus file RAW
        rm -f "$file"
        if test "$is_verbose" = "1"
            echo "[HAPUS]    $file (File pasangan JPG sudah ada, menghapus RAW)"
        end
    end
    exit 0
end

# ==============================================================================
# 2. WORKER MODE: TAHAP 2 (JPG → AVIF)
# ==============================================================================
if test "$argv[1]" = "--worker"
    set file "$argv[2]"
    set is_verbose "$argv[3]"

    if not test -f "$file"
        exit 0
    end

    set dir (dirname "$file")
    set base (string replace -r '\.[^.]+$' '' (basename "$file"))

    set final_avif "$dir/$base.avif"
    set tmp_file "$dir/$base.avif.tmp"

    # Bersihkan file .tmp jika worker terkena Ctrl+C di tengah proses encoding
    function cleanup_avif --on-signal INT --on-signal TERM --inherit-variable tmp_file
        rm -f "$tmp_file"
        exit 130
    end

    # 1. Cek apakah file JPG sudah pernah ditandai oleh script sebelumnya
    set existing_tag (exiftool -s3 -UserComment "$file" 2>/dev/null)
    if string match -q "*COMPRESSED_BY_SCRIPT*" "$existing_tag"
        if test "$is_verbose" = "1"
            echo "[SKIP-TAG] $file (Sudah memiliki metadata kompresi)"
        end
        exit 0
    end

    # 2. Konversi JPG ke AVIF (yuv 420, speed 8, q60)
    avifenc "$file" "$tmp_file" -q 60 --speed 8 --yuv 420 -j 1 >/dev/null 2>&1

    if test -s "$tmp_file"
        set comp_date (date -Iseconds)
        set host_name (hostname)
        set meta_payload "COMPRESSED_BY_SCRIPT=true;DATE=$comp_date;HOST=$host_name"

        # 3. Salin EXIF & suntikkan tag metadata ke file .tmp
```

```
        exiftool -overwrite_original -TagsFromFile "$file" \
            -UserComment="$meta_payload" \
            -XMP:CreatorTool="fish-avif-compressor ($host_name)" \
            "$tmp_file" >/dev/null 2>&1

        set orig_size (stat -c %s "$file")
        set new_size (stat -c %s "$tmp_file")

        # 4. Bandingkan ukuran
        if test $new_size -lt $orig_size
            touch -r "$file" "$tmp_file"
            mv "$tmp_file" "$final_avif"
            rm -f "$file"
            if test "$is_verbose" = "1"
                set orig_kb (math -s 1 "$orig_size / 1024")
                set new_kb (math -s 1 "$new_size / 1024")
                set diff_pct (math -s 1 "($orig_size - $new_size) / $orig_size * 100")
                echo "[OK-AVIF]  $file → $final_avif ($orig_kb KB → $new_kb KB, hemat $diff_pct%)"
            end
        else
            rm -f "$tmp_file"
            exiftool -P -overwrite_original -UserComment="$meta_payload;STATUS=SKIPPED_LARGER" "$file" >/dev/null 2>&1
            if test "$is_verbose" = "1"
                set orig_kb (math -s 1 "$orig_size / 1024")
                set new_kb (math -s 1 "$new_size / 1024")
                echo "[BATAL]    $file (AVIF $new_kb KB ≥ JPG $orig_kb KB, JPG asli dipertahankan)"
            end
        end
    else
        rm -f "$tmp_file"
        if test "$is_verbose" = "1"
            echo "[GAGAL]    $file (Gagal encode ke AVIF)"
        end
    end
    exit 0
end

# ==============================================================================
# 3. MAIN MODE: Parsing Argumen, State Management, & Pengolahan Paralel
# ==============================================================================

# Parsing argumen CLI menggunakan builtin argparse Fish
argparse 'f/fresh' 'v/verbose' 'h/help' -- $argv
or exit 1

if set -q _flag_help
    echo "Penggunaan: "(status filename)" [OPSI]"
    echo ""
    echo "Opsi:"
    echo "  -f, --fresh     Mulai pemindaian dari awal (abaikan/hapus log progres sebelumnya)"
    echo "  -v, --verbose   Tampilkan log detail untuk setiap file yang diproses"
    echo "  -h, --help      Tampilkan pesan bantuan ini"
    exit 0
end

# Set variabel flag untuk dikirim ke worker
set verbose_flag 0
if set -q _flag_verbose
    set verbose_flag 1
end

# Cek Ketersediaan Dependensi
for cmd in avifenc parallel exiftool magick stat date hostname
    if not command -q $cmd
        echo "Error: Perintah '$cmd' tidak ditemukan. Silakan install terlebih dahulu."
        exit 1
    end
end

set self_script (status filename)

# Direktori tersembunyi untuk menyimpan state progres
set state_dir ".avif_compress_state"
set raw_list "$state_dir/raw_queue.txt"
set raw_log "$state_dir/raw_joblog.txt"
set jpg_list "$state_dir/jpg_queue.txt"
set jpg_log "$state_dir/jpg_joblog.txt"
set init_size_file "$state_dir/size_before.txt"

# Jika dipanggil dengan --fresh (atau -f), hapus state progres sebelumnya
if set -q _flag_fresh
    if test -d "$state_dir"
        echo "[FRESH MODE] Menghapus log progres sebelumnya dan memulai pemindaian baru..."
        rm -rf "$state_dir"
    else
        echo "[FRESH MODE] Memulai pemindaian baru dari awal..."
    end
end

# Bersihkan sisa file .tmp yang mungkin tertinggal dari sesi crash sebelumnya
find . -type f \( -name "*.avif.tmp" -o -name "*.jpg.tmp" \) -delete 2>/dev/null

echo "≡≡≡ Memulai Otomasi Konversi Gambar ke AVIF ≡≡≡"
if test "$verbose_flag" = "1"
    echo "[VERBOSE MODE] Output detail per file diaktifkan."
end

if test -d "$state_dir"
    echo "[RESUME MODE] Progres sesi sebelumnya ditemukan! Melanjutkan dari titik terakhir..."
    echo "(Gunakan argumen '--fresh' jika isi folder telah berubah dan ingin scan ulang)"
else
    mkdir -p "$state_dir"
end
echo ""

# Catat Ukuran Folder Awal
if not test -f "$init_size_file"
    du -sb --exclude="$state_dir" . | string split -f1 \t > "$init_size_file"
end
set size_before_b (cat "$init_size_file")
```

```
# Tangkap sinyal Ctrl+C di Main Process
function on_main_interrupt --on-signal INT --on-signal TERM --inherit-variable state_dir
    echo ""
    echo "================================================================="
    echo "[DIBERHENTIKAN] Proses dihentikan oleh pengguna (Ctrl+C)."
    echo "Progres telah disimpan di '$state_dir'."
    echo "Jalankan ulang script ini untuk lanjut, atau tambahkan '--fresh' untuk ulang dari awal."
    echo "================================================================="
    exit 130
end

# ------------------------------------------------------------------------------
# TAHAP 1: Konversi Seluruh File RAW ke JPG
# ------------------------------------------------------------------------------
if not test -f "$state_dir/stage1.done"
    if not test -f "$raw_list"
        echo "==> [Tahap 1/2] Memindai daftar file RAW..."
        find . -type f \( \
            -iname "*.cr2" -o -iname "*.cr3" -o \
            -iname "*.arw" -o -iname "*.srf" -o -iname "*.sr2" -o \
            -iname "*.nef" -o -iname "*.nrw" -o \
            -iname "*.dng" -o -iname "*.raf" -o \
            -iname "*.rw2" -o -iname "*.orf" -o -iname "*.pef" \
        \) > "$raw_list"
    else
        echo "==> [Tahap 1/2] Memuat daftar file RAW dari sesi sebelumnya..."
    end

    set raw_count (wc -l < "$raw_list" | string trim)
    if test "$raw_count" -gt 0
        echo "==> [Tahap 1/2] Memproses $raw_count File RAW ke JPG..."
        parallel --bar -j 100% --joblog "$raw_log" --resume \
            fish "$self_script" --worker-raw {} "$verbose_flag" :::: "$raw_list"

        if test $status -ne 0
            on_main_interrupt
        end
    else
        echo "==> [Tahap 1/2] Tidak ada file RAW yang ditemukan."
    end

    touch "$state_dir/stage1.done"
else
    echo "==> [Tahap 1/2] Sudah selesai pada sesi sebelumnya. Melewati..."
end

# ------------------------------------------------------------------------------
# TAHAP 2: Konversi Seluruh File JPG ke AVIF (Termasuk Output Tahap 1)
# ------------------------------------------------------------------------------
echo ""
if not test -f "$state_dir/stage2.done"
    if not test -f "$jpg_list"
        echo "==> [Tahap 2/2] Memindai daftar file JPG..."
        find . -type f \( -iname "*.jpg" -o -iname "*.jpeg" \) > "$jpg_list"
    else
        echo "==> [Tahap 2/2] Memuat daftar file JPG dari sesi sebelumnya..."
    end

    set jpg_count (wc -l < "$jpg_list" | string trim)
    if test "$jpg_count" -gt 0
        echo "==> [Tahap 2/2] Memproses $jpg_count File JPG ke AVIF..."
        parallel --bar -j 100% --joblog "$jpg_log" --resume \
            fish "$self_script" --worker {} "$verbose_flag" :::: "$jpg_list"

        if test $status -ne 0
            on_main_interrupt
        end
    else
        echo "==> [Tahap 2/2] Tidak ada file JPG yang ditemukan."
    end

    touch "$state_dir/stage2.done"
end

# ------------------------------------------------------------------------------
# LAPORAN RINGKASAN UKURAN FOLDER & BERSIHKAN STATE
# ------------------------------------------------------------------------------
set size_after_b (du -sb --exclude="$state_dir" . | string split -f1 \t)

# Hapus folder state karena seluruh pekerjaan sudah tuntas 100%
rm -rf "$state_dir"

set before_mb (math -s 2 "$size_before_b / 1024 / 1024")
set after_mb (math -s 2 "$size_after_b / 1024 / 1024")
set saved_mb (math -s 2 "$before_mb - $after_mb")

if test $size_before_b -gt 0
    set saved_pct (math -s 2 "($size_before_b - $size_after_b) / $size_before_b * 100")
else
    set saved_pct 0
end

echo ""
echo "========================================="
echo "        HASIL RINGKASAN KONVERSI         "
echo "========================================="
echo "Ukuran Folder Awal  : $before_mb MB"
echo "Ukuran Folder Akhir : $after_mb MB"
echo "Total Hemat Ukuran  : $saved_mb MB ($saved_pct%)"
echo "========================================="
```

Apa Saja yang Ditambahkan?

1. **Argumen `--fresh` (atau `-f`)**
    - Menghapus folder `.avif_compress_state` sebelum pemindaian dimulai, sehingga memaksa script mendata ulang seluruh isi direktori dengan `find`.
    - **Aman untuk file lama:** Meskipun di-reset dengan `--fresh`, file yang sudah terkompresi atau JPG yang sebelumnya sudah ditandai `SKIPPED_LARGER` **tetap tidak akan dikompresi ulang** karena pengecekan tag metadata `COMPRESSED_BY_SCRIPT` di dalam file tetap berjalan.
2. **Argumen `-v` / `--verbose`**
    - Meneruskan flag ke setiap *worker* paralel untuk mencetak status eksekusi per file secara *real-time* di atas *progress bar* GNU Parallel (GNU Parallel secara otomatis menjaga *progress bar* tetap

berada di baris paling bawah).

- Contoh tampilan log saat `-v` aktif:
  - `[OK-RAW] ./IMG_001.CR2 → ./IMG_001.jpg (24500.2 KB → 6120.5 KB)`
  - `[OK-AVIF] ./IMG_001.jpg → ./IMG_001.avif (6120.5 KB → 840.2 KB, hemat 86.2%)`
  - `[BATAL] ./meme.jpg (AVIF 145.2 KB ⩾ JPG 120.0 KB, JPG asli dipertahankan)`
  - `[SKIP-TAG] ./meme.jpg (Sudah memiliki metadata kompresi)`
  - `[HAPUS] ./IMG_002.CR2 (File pasangan JPG sudah ada, menghapus RAW)`

3. **Fleksibilitas `argparse` & Bantuan (`-h` / `--help`)**
   - Kamu bisa memanggil argumen dalam urutan apa pun atau digabung:
     - `./compress.fish -v` (Lanjut/mulai dengan log detail)
     - `./compress.fish --fresh` (Ulang scan dari awal tanpa log detail)
     - `./compress.fish -fv` atau `./compress.fish --fresh -v` (Ulang scan dari awal + tampilkan log detail)

User prompt: tambahkan argumen `--fast` . aku notice setelah kita menambahkan fitur pengecekan metadata di awal, itu scanning jadi lebih lambat. jadi kalo pake `--fast` dia bakal skip pengecekan metadata kompresi di awal

Response: Penambahan argumen `--fast` telah diterapkan pada script di bawah ini. Saat `--fast` diaktifkan, setiap *worker* akan langsung melewati proses pembacaan tag `exiftool` di awal dan langsung melakukan konversi.

(Catatan kinerja: `exiftool` ditulis menggunakan Perl, sehingga memanggilnya satu per satu untuk mengecek ribuan file memang memiliki overhead startup sekitar 50-100 ms per file. Dengan `--fast`, pemanggilan `exiftool` di awal tiap file ditiadakan sehingga proses langsung masuk ke `magick` atau `avifenc`).

```
#!/usr/bin/env fish

# ==============================================================================
# 1. WORKER MODE: TAHAP 1 (RAW → JPG)
# ==============================================================================
if test "$argv[1]" = "--worker-raw"
    set file "$argv[2]"
    set is_verbose "$argv[3]"
    set is_fast "$argv[4]"

    # Jika file sudah tidak ada (misal terhapus di sesi sebelumnya), lewati
    if not test -f "$file"
        exit 0
    end

    set dir (dirname "$file")
    set base (string replace -r '\.[^.]+$' '' (basename "$file"))
    set target_jpg "$dir/$base.jpg"
    set tmp_jpg "$dir/$base.jpg.tmp"

    # Bersihkan file .tmp jika worker dihentikan paksa (Ctrl+C / SIGINT / SIGTERM)
    function cleanup_raw --on-signal INT --on-signal TERM --inherit-variable tmp_jpg
        rm -f "$tmp_jpg"
        exit 130
    end

    # Cek apakah file RAW sudah memiliki tag kompresi (Dilewati jika --fast aktif)
    if test "$is_fast" ≠ "1"
        set existing_tag (exiftool -s3 -UserComment "$file" 2>/dev/null)
        if string match -q "*COMPRESSED_BY_SCRIPT*" "$existing_tag"
            if test "$is_verbose" = "1"
                echo "[SKIP-TAG] $file (Sudah pernah diproses sebelumnya)"
            end
            exit 0
        end
    end

    # Cek apakah JPG sudah ada (untuk kasus RAW + JPG mode)
    if not test -f "$target_jpg"; and not test -f "$dir/$base.JPG"
        # Render ke file .tmp terlebih dahulu agar aman jika di-Ctrl+C
        magick "$file" -quality 98 "$tmp_jpg" >/dev/null 2>&1

        if test -s "$tmp_jpg"
            exiftool -overwrite_original -TagsFromFile "$file" "$tmp_jpg" >/dev/null 2>&1

            set orig_size (stat -c %s "$file")
            set new_size (stat -c %s "$tmp_jpg")

            if test $new_size -lt $orig_size
                touch -r "$file" "$tmp_jpg"
                mv "$tmp_jpg" "$target_jpg"
                rm -f "$file"
                if test "$is_verbose" = "1"
                    set orig_kb (math -s 1 "$orig_size / 1024")
                    set new_kb (math -s 1 "$new_size / 1024")
                    echo "[OK-RAW]   $file → $target_jpg ($orig_kb KB → $new_kb KB)"
                end
            else
                rm -f "$tmp_jpg"
                set comp_date (date -Iseconds)
                set host_name (hostname)
                exiftool -P -overwrite_original -UserComment="COMPRESSED_BY_SCRIPT=true;DATE=$comp_date;HOST=$host_name;STATUS=SKIPPED_LARGER" "$file" >/dev/null 2>&1
                if test "$is_verbose" = "1"
                    set orig_kb (math -s 1 "$orig_size / 1024")
                    set new_kb (math -s 1 "$new_size / 1024")
                    echo "[BATAL]    $file (JPG $new_kb KB ⩾ RAW $orig_kb KB, dilewati)"
                end
            end
        else
            rm -f "$tmp_jpg"
            if test "$is_verbose" = "1"
                echo "[GAGAL]    $file (Gagal merender RAW ke JPG)"
            end
        end
    else
        # Jika JPG versi asli sudah ada, langsung hapus file RAW
        rm -f "$file"
        if test "$is_verbose" = "1"
            echo "[HAPUS]    $file (File pasangan JPG sudah ada, menghapus RAW)"
        end
    end
```

```
    exit 0
end

# ==============================================================================
# 2. WORKER MODE: TAHAP 2 (JPG → AVIF)
# ==============================================================================
if test "$argv[1]" = "--worker"
    set file "$argv[2]"
    set is_verbose "$argv[3]"
    set is_fast "$argv[4]"

    if not test -f "$file"
        exit 0
    end

    set dir (dirname "$file")
    set base (string replace -r '\.[^.]+$' '' (basename "$file"))

    set final_avif "$dir/$base.avif"
    set tmp_file "$dir/$base.avif.tmp"

    # Bersihkan file .tmp jika worker terkena Ctrl+C di tengah proses encoding
    function cleanup_avif --on-signal INT --on-signal TERM --inherit-variable tmp_file
        rm -f "$tmp_file"
        exit 130
    end

    # 1. Cek apakah file JPG sudah pernah ditandai oleh script sebelumnya (Dilewati jika --fast aktif)
    if test "$is_fast" ≠ "1"
        set existing_tag (exiftool -s3 -UserComment "$file" 2>/dev/null)
        if string match -q "*COMPRESSED_BY_SCRIPT*" "$existing_tag"
            if test "$is_verbose" = "1"
                echo "[SKIP-TAG] $file (Sudah memiliki metadata kompresi)"
            end
            exit 0
        end
    end

    # 2. Konversi JPG ke AVIF (yuv 420, speed 8, q60)
    avifenc "$file" "$tmp_file" -q 60 --speed 8 --yuv 420 -j 1 >/dev/null 2>&1

    if test -s "$tmp_file"
        set comp_date (date -Iseconds)
        set host_name (hostname)
        set meta_payload "COMPRESSED_BY_SCRIPT=true;DATE=$comp_date;HOST=$host_name"

        # 3. Salin EXIF & suntikkan tag metadata ke file .tmp
        exiftool -overwrite_original -TagsFromFile "$file" \
            -UserComment="$meta_payload" \
            -XMP:CreatorTool="fish-avif-compressor ($host_name)" \
            "$tmp_file" >/dev/null 2>&1

        set orig_size (stat -c %s "$file")
        set new_size (stat -c %s "$tmp_file")

        # 4. Bandingkan ukuran
        if test $new_size -lt $orig_size
            touch -r "$file" "$tmp_file"
            mv "$tmp_file" "$final_avif"
            rm -f "$file"
            if test "$is_verbose" = "1"
                set orig_kb (math -s 1 "$orig_size / 1024")
                set new_kb (math -s 1 "$new_size / 1024")
                set diff_pct (math -s 1 "($orig_size - $new_size) / $orig_size * 100")
                echo "[OK-AVIF]  $file → $final_avif ($orig_kb KB → $new_kb KB, hemat $diff_pct%)"
            end
        else
            rm -f "$tmp_file"
            exiftool -P -overwrite_original -UserComment="$meta_payload;STATUS=SKIPPED_LARGER" "$file" >/dev/null 2>&1
            if test "$is_verbose" = "1"
                set orig_kb (math -s 1 "$orig_size / 1024")
                set new_kb (math -s 1 "$new_size / 1024")
                echo "[BATAL]    $file (AVIF $new_kb KB ≥ JPG $orig_kb KB, JPG asli dipertahankan)"
            end
        end
    else
        rm -f "$tmp_file"
        if test "$is_verbose" = "1"
            echo "[GAGAL]    $file (Gagal encode ke AVIF)"
        end
    end
    exit 0
end

# ==============================================================================
# 3. MAIN MODE: Parsing Argumen, State Management, & Pengolahan Paralel
# ==============================================================================

# Parsing argumen CLI menggunakan builtin argparse Fish
argparse 'f/fresh' 'v/verbose' 'fast' 'h/help' -- $argv
or exit 1

if set -q _flag_help
    echo "Penggunaan: "(status filename)" [OPSI]"
    echo ""
    echo "Opsi:"
    echo "  -f, --fresh     Mulai pemindaian dari awal (abaikan/hapus log progres sebelumnya)"
    echo "      --fast      Lewati pengecekan metadata kompresi di awal (proses jauh lebih cepat)"
    echo "  -v, --verbose   Tampilkan log detail untuk setiap file yang diproses"
    echo "  -h, --help      Tampilkan pesan bantuan ini"
    exit 0
end

# Set variabel flag untuk dikirim ke worker
set verbose_flag 0
if set -q _flag_verbose
    set verbose_flag 1
end

set fast_flag 0
if set -q _flag_fast
```

```
    set fast_flag 1
end

# Cek Ketersediaan Dependensi
for cmd in avifenc parallel exiftool magick stat date hostname
    if not command -q $cmd
        echo "Error: Perintah '$cmd' tidak ditemukan. Silakan install terlebih dahulu."
        exit 1
    end
end

set self_script (status filename)

# Direktori tersembunyi untuk menyimpan state progres
set state_dir ".avif_compress_state"
set raw_list "$state_dir/raw_queue.txt"
set raw_log "$state_dir/raw_joblog.txt"
set jpg_list "$state_dir/jpg_queue.txt"
set jpg_log "$state_dir/jpg_joblog.txt"
set init_size_file "$state_dir/size_before.txt"

# Jika dipanggil dengan --fresh (atau -f), hapus state progres sebelumnya
if set -q _flag_fresh
    if test -d "$state_dir"
        echo "[FRESH MODE] Menghapus log progres sebelumnya dan memulai pemindaian baru..."
        rm -rf "$state_dir"
    else
        echo "[FRESH MODE] Memulai pemindaian baru dari awal..."
    end
end

# Bersihkan sisa file .tmp yang mungkin tertinggal dari sesi crash sebelumnya
find . -type f \( -name "*.avif.tmp" -o -name "*.jpg.tmp" \) -delete 2>/dev/null

echo "≡≡ Memulai Otomasi Konversi Gambar ke AVIF ≡≡"
if test "$verbose_flag" = "1"
    echo "[VERBOSE MODE] Output detail per file diaktifkan."
end
if test "$fast_flag" = "1"
    echo "[FAST MODE] Pengecekan tag metadata di awal dinonaktifkan."
end

if test -d "$state_dir"
    echo "[RESUME MODE] Progres sesi sebelumnya ditemukan! Melanjutkan dari titik terakhir..."
    echo "(Gunakan argumen '--fresh' jika isi folder telah berubah dan ingin scan ulang)"
else
    mkdir -p "$state_dir"
end
echo ""

# Catat Ukuran Folder Awal
if not test -f "$init_size_file"
    du -sb --exclude="$state_dir" . | string split -f1 \t > "$init_size_file"
end
set size_before_b (cat "$init_size_file")

# Tangkap sinyal Ctrl+C di Main Process
function on_main_interrupt --on-signal INT --on-signal TERM --inherit-variable state_dir
    echo ""
    echo "================================================================="
    echo "[DIBERHENTIKAN] Proses dihentikan oleh pengguna (Ctrl+C)."
    echo "Progres telah disimpan di '$state_dir'."
    echo "Jalankan ulang script ini untuk lanjut, atau tambahkan '--fresh' untuk ulang dari awal."
    echo "================================================================="
    exit 130
end

# ------------------------------------------------------------------------------
# TAHAP 1: Konversi Seluruh File RAW ke JPG
# ------------------------------------------------------------------------------
if not test -f "$state_dir/stage1.done"
    if not test -f "$raw_list"
        echo "⟹ [Tahap 1/2] Memindai daftar file RAW..."
        find . -type f \( \
            -iname "*.cr2" -o -iname "*.cr3" -o \
            -iname "*.arw" -o -iname "*.srf" -o -iname "*.sr2" -o \
            -iname "*.nef" -o -iname "*.nrw" -o \
            -iname "*.dng" -o -iname "*.raf" -o \
            -iname "*.rw2" -o -iname "*.orf" -o -iname "*.pef" \
        \) > "$raw_list"
    else
        echo "⟹ [Tahap 1/2] Memuat daftar file RAW dari sesi sebelumnya..."
    end

    set raw_count (wc -l < "$raw_list" | string trim)
    if test "$raw_count" -gt 0
        echo "⟹ [Tahap 1/2] Memproses $raw_count File RAW ke JPG..."
        parallel --bar -j 100% --joblog "$raw_log" --resume \
            fish "$self_script" --worker-raw {} "$verbose_flag" "$fast_flag" :::: "$raw_list"

        if test $status -ne 0
            on_main_interrupt
        end
    else
        echo "⟹ [Tahap 1/2] Tidak ada file RAW yang ditemukan."
    end

    touch "$state_dir/stage1.done"
else
    echo "⟹ [Tahap 1/2] Sudah selesai pada sesi sebelumnya. Melewati..."
end

# ------------------------------------------------------------------------------
# TAHAP 2: Konversi Seluruh File JPG ke AVIF (Termasuk Output Tahap 1)
# ------------------------------------------------------------------------------
echo ""
if not test -f "$state_dir/stage2.done"
    if not test -f "$jpg_list"
        echo "⟹ [Tahap 2/2] Memindai daftar file JPG..."
        find . -type f \( -iname "*.jpg" -o -iname "*.jpeg" \) > "$jpg_list"
    else
```

```
        echo "==> [Tahap 2/2] Memuat daftar file JPG dari sesi sebelumnya..."
    end

    set jpg_count (wc -l < "$jpg_list" | string trim)
    if test "$jpg_count" -gt 0
        echo "==> [Tahap 2/2] Memproses $jpg_count File JPG ke AVIF..."
        parallel --bar -j 100% --joblog "$jpg_log" --resume \
            fish "$self_script" --worker {} "$verbose_flag" "$fast_flag" :::: "$jpg_list"

        if test $status -ne 0
            on_main_interrupt
        end
    else
        echo "==> [Tahap 2/2] Tidak ada file JPG yang ditemukan."
    end

    touch "$state_dir/stage2.done"
end

# ------------------------------------------------------------------------------
# LAPORAN RINGKASAN UKURAN FOLDER & BERSIHKAN STATE
# ------------------------------------------------------------------------------
set size_after_b (du -sb --exclude="$state_dir" . | string split -f1 \t)

# Hapus folder state karena seluruh pekerjaan sudah tuntas 100%
rm -rf "$state_dir"

set before_mb (math -s 2 "$size_before_b / 1024 / 1024")
set after_mb (math -s 2 "$size_after_b / 1024 / 1024")
set saved_mb (math -s 2 "$before_mb - $after_mb")

if test $size_before_b -gt 0
    set saved_pct (math -s 2 "($size_before_b - $size_after_b) / $size_before_b * 100")
else
    set saved_pct 0
end

echo ""
echo "========================================="
echo "        HASIL RINGKASAN KONVERSI         "
echo "========================================="
echo "Ukuran Folder Awal  : $before_mb MB"
echo "Ukuran Folder Akhir : $after_mb MB"
echo "Total Hemat Ukuran  : $saved_mb MB ($saved_pct%)"
echo "========================================="
```

Detail Perubahan `--fast`

1. **Melewati Pembacaan EXIF Awal (`$is_fast` = 1)**: Saat `--fast` diberikan, script tidak lagi memanggil `exiftool -s3 -UserComment` di awal setiap worker. Ini memangkas hingga 50% jumlah pemanggilan `exiftool` secara keseluruhan.
2. **Tetap Menyuntikkan Metadata di Akhir**: Meskipun pengecekan di awal dilewati, file hasil kompresi `.avif` (atau file `.jpg` yang batal karena ukurannya lebih besar) **tetap diberikan tag metadata** `COMPRESSED_BY_SCRIPT`. Dengan begitu, jika suatu saat kamu menjalankan script *tanpa* `--fast`, file-file tersebut tetap bisa dikenali dengan baik.
3. **Kombinasi Bebas**: Bisa digabungkan dengan argumen lain seperti `./script.fish --fresh --fast -v`.

User prompt: cara mnegatasi /run/media/raihanlazuardi/ExternalHDD/Photos/PONDOK 0s 08:50:10 ❯ gasinafiv -v fish: Unknown command: gasinafiv /run/media/raihanlazuardi/ExternalHDD/Photos/PONDOK 0.01s 08:50:28 |127 ❯ padahal sudah ada di folder fish functions

Response: Error `fish: Unknown command: gasinafiv` meskipun filenya sudah ditaruh di `~/.config/fish/functions/` biasanya disebabkan oleh **3 hal utama**: aturan *autoloading* Fish yang mewajibkan deklarasi `function`, kesalahan ketik nama file (`afiv` vs `avif`), atau masalah ketika GNU Parallel mencoba memanggil ulang script tersebut.

Berikut adalah penyebab dan cara mengatasinya:

1. Masalah Utama: Script ini Memanggil Dirinya Sendiri (`parallel fish "$self_script"`)

Di dalam script kita sebelumnya, terdapat baris:

```
set self_script (status filename)
parallel ... fish "$self_script" --worker {}
```

Jika kamu menaruh kode tersebut di `~/.config/fish/functions/gasinafiv.fish` dan membungkusnya dengan `function gasinafiv ... end`, maka ketika `parallel` menjalankan `fish ~/.config/fish/functions/gasinafiv.fish --worker`, **tidak akan terjadi apa-apa** karena di dalam file `functions/`, Fish hanya mendefinisikan fungsi tanpa mengeksekusinya.

Solusi Terbaik (Pindahkan ke `~/.local/bin`): Karena ini adalah *standalone script* (bukan sekadar fungsi wrapper shell), tempat yang tepat adalah di folder binary lokal, bukan di `functions`:

1. Buat folder `~/.local/bin` jika belum ada, lalu pindahkan file script ke sana (tanpa ekstensi `.fish` agar bisa dipanggil langsung):

   ```
   mkdir -p ~/.local/bin
   mv ~/.config/fish/functions/gasinafiv* ~/.local/bin/gasinafiv
   ```

2. Berikan izin eksekusi (*executable permission*):

   ```
   chmod +x ~/.local/bin/gasinafiv
   ```

3. Pastikan `~/.local/bin` sudah masuk ke `$PATH` Fish kamu (cukup jalankan sekali seumur hidup):

   ```
   fish_add_path ~/.local/bin
   ```

Sekarang kamu bisa langsung memanggil `gasinafiv -v` dari folder mana saja, dan `parallel` di dalamnya juga akan bekerja dengan normal.

2. Jika Tetap Ingin Menaruhnya di `~/.config/fish/functions/`

Di Fish, folder `functions/` menganut sistem **Autoloading ketat**:

- Nama file **harus sama persis** dengan perintah yang diketik (misal: `~/.config/fish/functions/gasinafiv.fish`). *Cek juga apakah kamu tidak sengaja menamainya `gasinavif.fish` (avif) tapi mengetik `gasinafiv` (afiv).*
- Isi di dalam file tersebut **wajib** dibuka dengan `function nama_perintah` dan ditutup dengan `end`. Jika langsung berisi `#!/usr/bin/env fish` tanpa blok `function`, Fish akan mengabaikannya.

Agar tetap bisa jalan di dalam `functions/` **sekaligus** tetap bisa dipanggil oleh `parallel` sebagai file script mandiri, tambahkan blok pendek di bagian **paling atas** dan **paling bawah** file `~/.config/fish/functions/gasinafiv.fish` seperti ini:

```
#!/usr/bin/env fish

function gasinafiv
    # ========================================================
    # (TEMPEL SELURUH ISI SCRIPT DARI ATAS SAMPAI BAWAH DI SINI)
    # ========================================================
end

# Baris ini penting: Jika file ini dieksekusi langsung oleh GNU Parallel
# sebagai script mandiri (bukan di-autoload oleh interaktif shell),
# otomatis jalankan fungsinya dengan meneruskan semua argumen ($argv):
if status is-command-substitution; or not status is-interactive
```

```
    gasinafiv $argv
end
```

(Catatan kecil: jika menggunakan cara ke-2 di dalam `functions/`, ubah juga semua perintah `exit 0` atau `exit 1` di dalam script menjadi `return 0` atau `return 1` agar saat selesai dijalankan, terminal kamu tidak ikut tertutup).
